Preemption-bounded Consistency Testing

Malay Kedia

March 2025

IIT Bombay, CSE Department

Outline

NP-completeness for all models

Fine-Grained Complexity Analysis

- RA, SRA and Relaxed
- WRA and SC

W[1]-hardness with RMW

Poly Time Results for Single-Writer Programs

Motivation

We saw in class :
In general, consistency checking problem is NP-complete for SC, and other weak models.

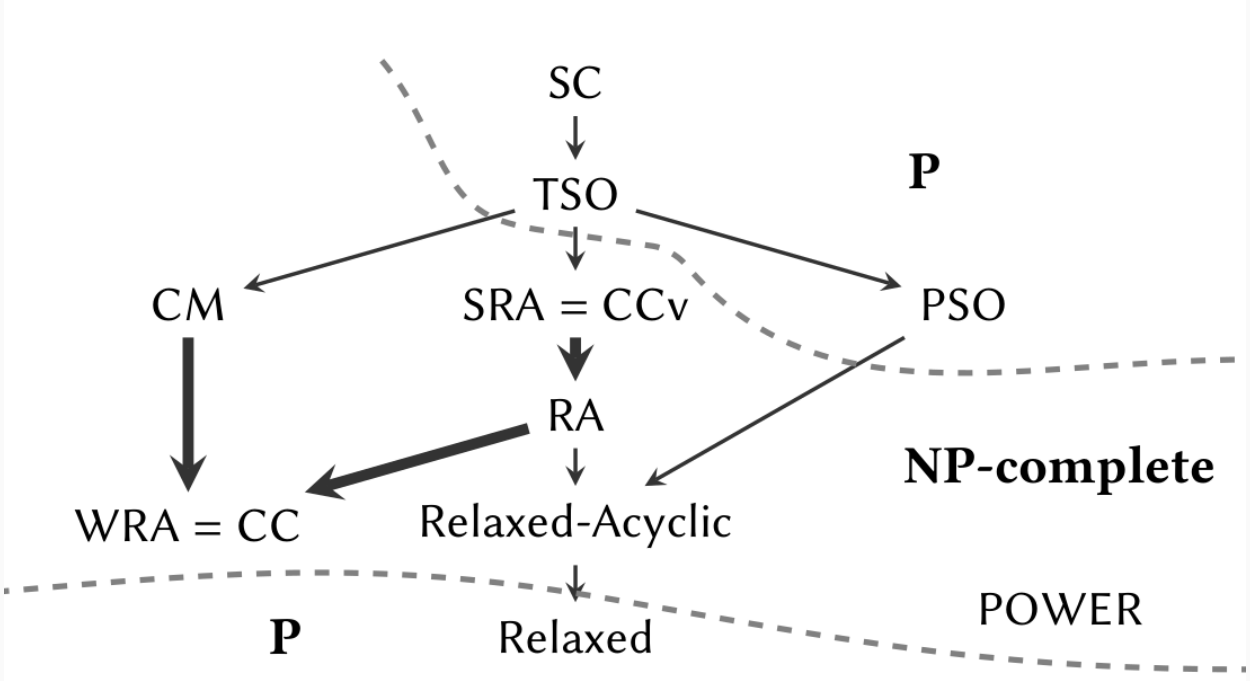
Motivation

What if we bound some quantity in the run?

Say number of threads, memory locations, or values.

Motivation

What if we bound some quantity in the run?
Say number of threads, memory locations, or values.



Consistency testing for bounded inputs is NP-complete for most weak models, even in their RMW-free fragment.

Refer to "How Hard is Weak-Memory Testing?" by Soham Chakraborty et al. for more details.

Now What?

We know, a lot of bugs in concurrent programs can be caught using runs with a small number of preemptions.

Is consistency checking problem under bounded no. of preemptions easier?
Turns out, NO

Overview

What all does the paper cover?

1. The preemption-bounded consistency problem is NP-complete for all memory models. (Technically done in class already)
2. For some weak models, it shows a stronger lower bound under the Exponential Time Hypothesis (ETH).
3. It shows W[1]-hardness across all models, with the parameter being the number of threads when we include rmw statements.
4. For single-writer programs, it shows $\mathcal{O}(n)$ for some models, and $\mathcal{O}(n^2)$ for others. Additionally, for a subset of the latter memory models, it shows a lower bound under the Boolean Matrix Multiplication (BMM) hypothesis.

NP-completeness for all models

The preemption-bounded consistency testing problem is NP-complete for all memory models, even when the number of preemptions is fixed.

Recap

We have already seen the range reduction for NP-hardness of consistency checking for WRA, RA, SRA in class.

Just one point to note, the reduction we saw works even if we restrict programs to 0-preemption.

Fine-Grained Complexity Analysis

We show that for C11 models other than WRA (in their rmw-free fragments), the 0-preemption-bounded consistency problem does not have a $2^{o(k \log k)} \cdot n^{O(1)}$ time procedure, where k is the number of threads and n is the number of events.

Note that this result implies NP-hardness.

Permutation SAT

$\{x_1, x_2, \ldots, x_k\}$ are arbitrary variables.
Order-literal: $x_i < x_j$ between two distinct variables.

An (ℓ, p)-Permutation SAT formula contains up to p order-literals in a clause, and each order literal is of the form $x_{i_1} < x_{i_2} < \cdots < x_{i_\ell}$.

A $(2, 4)$-Permutation-SAT formula φ is a conjunction $\{C_i\}_{i \in [m]}$ of m 4-clauses. The 2 in $(2, 4)$ denotes that each order literal contains 2 variables.

Satisfiability of Permutation SAT

A Permutation-SAT formula φ is said to be satisfiable if there is a total ordering on the variables such that in every clause, at least one of the order-literals is satisfied.

Exponential Time Hypothesis (ETH): There is no $2^{o(n)}$ algorithm for 3-CNF-SAT.

Assuming ETH, we get the following theorem:

Theorem

There is no $2^{o(k \log k)} n^{\mathcal{O}(1)}$ algorithm for checking satisfiability of $(2, 4)$-Permutation SAT formulas with n clauses and k variables.

Reduction

For a permutation SAT formula with k variables and n clauses, we construct an execution graph with $\mathcal{O}(k^2)$ threads and $\mathcal{O}(n)$ events in $\mathcal{O}(n + k^2)$ time.
Now, if there were a $2^{o(k \log k)} \cdot n^{O(1)}$ time procedure for 0-preemption-bounded consistency problem, the formula φ can be reduced to the consistency of our execution graph and solved in

$$\mathcal{O}(n + k^2) \cdot 2^{o(\sqrt{k^2} \log k^2)} \cdot n^{\mathcal{O}(1)} \equiv 2^{o(k \log k)} \cdot n^{\mathcal{O}(1)}$$

Contradiction, so, no such $2^{o(k \log k)} \cdot n^{O(1)}$ time procedure exists.

The Construction

Let $\varphi = \{C_i\}_{i \in [m]}$ be a (2,4)-Permutation SAT formula over k variables $\{x_j\}_{j \in [k]}$ and m clauses of the form $C_i = (A_{in})_{1 \leq i \leq m,\, 1 \leq n \leq 4}$, where each A_{in} is an order-literal.

We construct a partial execution graph $\overline{X} = (E, \mathsf{po})$ with $k^2 + k + 2$ threads, accessing $m + 2$ shared variables $x, c_1, \ldots, c_m, d$ and $2k^2 + 1$ values $\{\top_{i,j}, \bot_{i,j}\}_{1 \leq i,j \leq k}, \top$

such that φ is satisfiable iff $\overline{X} \models$ Relaxed, RA, SRA with 0 preemptions

The Construction

$\top_{i,j}$ means that $x_i < x_j$ is true, and $\bot_{i,j}$ means that $x_i < x_j$ is false.

Thread P_i says that x_i is the largest variable.

If threads are executed in order $P_{i_1}, P_{i_2}, \ldots, P_{i_k}$,
we induce the ordering $x_{i_1} < x_{i_2} < \cdots < x_{i_k}$

Thread $T_{i,j}$ checks $x_i < x_j$ & writes to the clauses in which this appears.

Actual Construction Done on Board

Why does this work??

Trust me, it does!!

Other Results

Theorem

When no. of preemptions c is fixed, the preemption-bounded consistency problem for SC *is fixed-parameter tractable with parameter k. Moreover, when c is fixed, there is no $2^{o(\sqrt{k}\log k)} \cdot n^{\mathcal{O}(1)}$ algorithm for the problem.*

Theorem

Assuming ETH, there is no $2^{o(\sqrt{k}\log k)} \cdot \mathcal{O}(n)$ time procedure for the k-preemption bounded consistency testing problem under WRA, *where k is the number of threads and n is the number of events in the program.*

W[1]-hardness with RMW

What is FPT?

In general, assuming P$\neq$NP, NP-complete problems are intractable as they require time that is exponential in the total size of the input.

What is FPT?

In general, assuming P$\neq$NP, NP-complete problems are intractable as they require time that is exponential in the total size of the input.

Some problems can be solved by algorithms that are exponential only in the size of a fixed parameter while polynomial in the size of the input.

An algorithm is fixed-parameter tractable (FPT) if it runs in time exponential only in a fixed parameter and polynomial in the input size, enabling efficient solutions when the parameter is small.

What is W[1]?

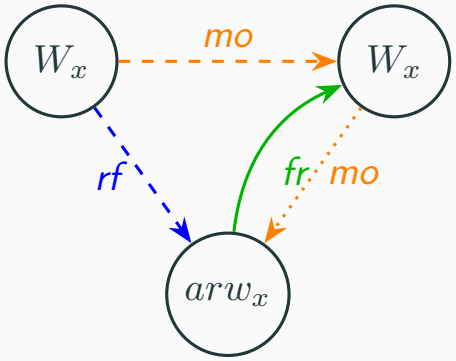
W[1] is a class of parameterized problems believed to be outside FPT, much like NP-complete problems are believed to be outside P.

A canonical W[1]-complete problem is the k-Independant Set problem: determining whether a graph contains a independant set of size k.

With atomic-read-modify-write instructions, the preemption-bounded consistency testing problem is W[1]-hard for all memory models.

Axioms for RMW

When `rmw` operations are allowed, consistency is enforced by atomicity, which enforces $\texttt{irr(fr; mo}_\texttt{x}\texttt{)}$ for each variable x.



This enforces that no write w' can take place between the read $\texttt{r}(x, u)$ and write $\texttt{w}(x, v)$ of a $\texttt{rmw}(x, u, v)$ operation.

Atomicity is an axiom for all models except WRA where `mo` is not there. For WRA we have a weaker form of atomicity which enforces that for two `rmw` operations u_1, u_2, if $w_1 \xrightarrow{\texttt{rf}} u_1$ and $w_2 \xrightarrow{\texttt{rf}} u_2$, then $w_1 \neq w_2$.

Reduction: Consistency Checking $\leq$ k-ind set problem

Given $G = (V, E)$ and a constant k, we construct a partial execution graph $\overline{X}_G$ with $k+1$ threads, $|E| + k$ shared variables and 5 values $\{*, \top_\perp, \top_1, \perp_1, \perp\}$ such that G has an independent set of size k iff $\overline{X}_G$ can be extended to an SC-consistent execution graph X_G with not more than $2k+1$ preemptions.

Since $X_G \models \mathtt{SC}$, the reduction works for all memory models. An edge in the graph G between i to j is encoded by the shared variable e_{ij} for $i < j$ (initialized to $\perp$). We also have k shared variables $x_1, \ldots, x_k$ which help with selecting the independent set of size k, if one exists.

Essentially, we have k selector threads, each with $|V|$ blocks, with i^{th} block trying to select v_i in the independant set. If all threads can simultaneously select k vertices so that no edge is shared between them, only then can the Checker thread proceed.

Actual construction done on Board

Poly Time Results for Single-Writer Programs

Results

Let $\bar{X} = \langle E, po \rangle$ be an execution graph for a single-writer program, where $n = |E|$, $k =$ number of threads.

0-preemption consistency can be tested in:

- $\mathcal{O}(n)$ time for $\mathcal{M} \in \{\text{SC}\}$
- $\mathcal{O}(n^2)$ time for $\mathcal{M} \in \{\text{SRA, RA, WRA, Relaxed}\}$

Algorithm Overview

Two main steps:

1. Build a graph G_{ord} whose vertices are threads $t_1, \ldots, t_k$ and for each shared variable x, add an edge $t_x \to t$ if thread t performs a read on x. In any consistent 0-preemption run, t_x must appear before t. Therefore, if G_{ord} contains a cycle, we conclude that there is no rf inducing a 0-preemption run on $\bar{X}$.
2. For every read, we build an rf relation where for each read, we find the earliest available write which does not violate the axioms of the considered model. If there is no matching write for a read, we conclude that there is no 0-preemption run.

Complexity

- In Step-1, we scan the entire graph once, which takes $\mathcal{O}(n)$ time. The graph has k vertices and at max n edges. So, detecting a cycle is $\mathcal{O}(k+n)$. Overall, complexity is $\mathcal{O}(n)$
- In Step-2, for each read $r(x, v)$, we need to find an appropriate write in t_x, which can be done by brute force in O(n) time. Hence, the overall complexity is $\mathcal{O}(n^2)$.
- For SC, in Step-2, each read on a variable x can only read the last value written by the writer t_x. This can be done in $\mathcal{O}(n)$.

Lower Bound for SRA, RA & WRA

Under BMM hypothesis, there is no $\mathcal{O}(n^{\frac{3}{2}-\varepsilon})$ algorithm for the 0-preemption consistency problem in single-writer programs with respect to the memory models SRA, RA, and WRA.

BMM Hypothesis: There is no $\mathcal{O}(n^{3-\epsilon}), \epsilon > 0$ algorithm for Boolean Matrix Multiplication.

Theorem

Under the BMM hypothesis, there is no $\mathcal{O}(n^{3-\varepsilon})$ algorithm for detecting a triangle in a graph with n vertices.

Reduction: Triangle Detection $\leq$ Consistency Checking

Given a graph $G = (V_G, E_G)$ with n vertices,
we construct an RMW-free partial execution $X = \langle E, \mathsf{po} \rangle$ with
$|E| = \mathcal{O}(|V_G| + |E_G|)$ such that X is consistent with 0 preemptions
under any of RA, WRA, and SRA if and only if G is triangle-free.

The Construction

- For each vertex $v \in V_G$, we introduce threads $t_v^a, t_v^b, t_v^c, t_v^d$.
- For each edge $(u, v) \in E_G$, we define memory locations a_{uv}, b_{uv}, c_{uv}.
- Additionally, for each vertex $v \in V_G$, we define locations a_v, b_v, c_v.

This construction yields:

- Total threads: $4|V_G|$
- Total memory locations: $3|V_G| + 3|E_G|$
- Total events: $5|V_G| + 2|E_G|$

$N_v = \{\alpha \mid (v, \alpha) \in E_G\}$

$W(x_{v\alpha}, 0), R(x_{v\alpha}, 0)$ are macros representing respectively, the sequence of events $\mathsf{w}(x_{v\alpha_1}, 0) \ldots \mathsf{w}(x_{v\alpha_m}, 0)$ and $\mathsf{r}(x_{v\alpha_1}, 0) \ldots \mathsf{r}(x_{v\alpha_m}, 0)$ such that $\alpha_1, \ldots, \alpha_n \in N_v$.

The required program is:

$$
\begin{aligned}
t_v^a &: \mathsf{w}(a_v, 0), \mathsf{w}(a_v, 1), W(a_{v\omega}, 0); \\
t_v^b &: R(a_{v\omega}, 0), W(b_v), W(b_{v\omega}, 0); \\
t_v^c &: R(b_{v\omega}, 0), W(c_v), W(c_{v\omega}, 0) \\
t_v^d &: R(c_{v\omega}, 0), \mathsf{r}(a_v, 0)
\end{aligned}
$$

The core idea

1. $(x, y) \in E_G$ iff $\mathsf{w}(a_x, 0) \xrightarrow{\texttt{po}} \mathsf{w}(a_x, 1) \xrightarrow{\texttt{hb}} \mathsf{w}(b_y)$
2. $(x, y) \in E_G$ iff $\mathsf{w}(b_x) \xrightarrow{\texttt{hb}} \mathsf{w}(c_y)$
3. $(x, y) \in E_G$ iff $\mathsf{w}(c_x) \xrightarrow{\texttt{hb}} \mathsf{r}(a_y, 0)$

The key invariant is that for any $v \in V_G$, $\mathsf{w}(a_v, 1) \xrightarrow{\texttt{hb}} \mathsf{r}(a_v, 0)$ iff G contains a triangle. The former violates (weak) read coherence, and would render $\overline{X}$ inconsistent under WRA (hence also under RA, SRA).

References

"Verifying Sequential Consistency under Bounded Preemptions" by R. Govind, S. Krishna, Sanchari Sil, B. Srivathsan

Thank You for Bearing with me !!